VIBE-CODED APPLICATION SECURITY HARDENING

20 vulnerability-focused implementation prompts + Git repository exposure remediation

Purpose
A practical prompt pack for directing an AI coding agent to inspect, fix, test and document security weaknesses in an existing web application. Designed for full-stack projects using modern frontend frameworks, APIs, authentication providers and databases such as Supabase or PostgreSQL.

Important
These prompts are engineering instructions, not a certification or guarantee of security. Run them against an authorized project, review generated code, test in staging, and obtain an independent security review before handling sensitive or high-value data.

Includes: SQL injection • prompt injection • RLS • mass assignment • privilege escalation • open redirects • command injection • audit logging • secrets • password hashing • slopsquatting • dependency CVEs • SRI • CSP • clickjacking • public storage • debug mode • token expiry • insecure deserialization • SSRF

How to use this prompt pack

Use one vulnerability prompt at a time. Give the coding agent access to the repository and ask it to inspect before editing. Do not paste production secrets into an AI chat. Require a concise change report, tests, and any manual configuration steps. For critical changes, review the diff before merging.

Recommended workflow

1. Create a separate branch and confirm the target environment is staging or local.
2. Ask the agent to map the affected routes, components, API handlers, database tables, storage buckets, auth flows and deployment configuration.
3. Require a threat model and reproduction test before changing code.
4. Implement the smallest safe fix; preserve intended product behavior.
5. Add automated regression tests, negative tests and authorization tests.
6. Run linting, type checks, unit tests, integration tests, dependency scans and build.
7. Review the complete diff; verify no secrets, test bypasses or debug switches were introduced.
8. Deploy to staging, execute the verification steps, then plan a controlled production release.

Universal instruction to prepend to every prompt

```
Act as a senior application security engineer. Work only in the authorized repository and environment I provide. First inspect the existing architecture and identify the exact affected files, routes, trust boundaries, data flows and configuration. Do not assume a framework, database or provider: detect it from the code. Do not break existing product behavior. Never expose, print, commit or request production secrets. Do not disable security controls to make tests pass. Implement defense in depth, fail securely, and follow current stable framework/provider guidance. Add regression tests, run relevant checks, report changed files, remaining risks, deployment/configuration steps and evidence from tests. If a safe fix requires a product decision or unavailable credential, stop and explain the blocker rather than inventing a value.
```

Security issue prompts

1. SQL injection

Risk: User-controlled data may alter a database query, allowing unauthorized reads, writes or deletion.

Copy/paste AI implementation prompt

```
Audit every database access path: API routes, server actions, RPC calls, search, filters, sorting, reporting and admin tools. Replace string-concatenated SQL with parameterized queries or the ORM/query builder's safe APIs. Allowlist dynamic identifiers such as sort columns and directions; parameters cannot safely represent identifiers. Validate input at the boundary, enforce least-privilege database roles, and ensure errors do not reveal query text or schema details. Review stored procedures and raw-query escape hatches. Add tests using quotes, comment markers, boolean payloads and time-delay-style strings as inert test input; verify they are treated only as data. Confirm authorization separately from injection prevention.
```

Required verification: Tests: parameterized-query regression tests; malformed-input tests; authorization tests; static scan for unsafe raw query construction. Do not test destructive payloads against production.

2. Prompt injection (LLM features)

Risk: Untrusted user text, documents, web pages or tool output may manipulate an AI model into ignoring instructions or misusing tools.

Copy/paste AI implementation prompt

```
Inventory all LLM prompts, retrieval sources, uploaded files, browser content and tool/function calls. Treat retrieved text and user content as untrusted data, never as system/developer instructions. Separate trusted instructions from quoted data; clearly delimit and label untrusted content. Do not rely on prompt wording as the only security boundary. Enforce authorization in application code before every tool call and data access. Give tools narrow schemas and least
```

```
privilege; validate arguments server-side; require explicit user confirmation for consequential actions. Prevent the model from directly executing arbitrary SQL, shell commands, payments, account changes or unrestricted URLs. Add output validation, sensitive-data filtering, rate/cost limits and audit records. Test direct and indirect injection in documents, images/OCR, retrieved pages and tool responses. Verify that secrets and cross-user data cannot be disclosed.
```

Required verification: Tests: adversarial prompt suite; tool authorization tests; cross-tenant data tests; action-confirmation tests. A model refusal alone is not proof of security.

3. Row Level Security (RLS) policies

Risk: Missing or incorrect row-level policies can expose one user's records to another user or allow unauthorized writes.

Copy/paste AI implementation prompt

```
For every tenant-owned table, inspect whether RLS is enabled and whether SELECT, INSERT, UPDATE and DELETE policies are correct. Define the ownership/tenant key and derive the authenticated identity from the trusted auth context, not from a client-supplied user_id. Apply policies for each operation; use both USING and WITH CHECK where appropriate. Review views, functions, RPCs, joins, storage metadata and service-role access because privileged paths may bypass RLS. Keep service-role credentials server-only. Deny by default, scope admin access explicitly, and avoid policies that accidentally allow all authenticated users. Add migrations and policy tests for anonymous, user A, user B, tenant admin and service role. Test reads and writes across tenants.
```

Required verification: Tests: prove user A cannot read, update, delete or insert rows for user B; verify anonymous denial; inspect privileged functions and storage policies.

4. Mass assignment

Risk: Binding an entire client request to a database model can let attackers change protected fields such as role, owner, price or account status.

Copy/paste AI implementation prompt

```
Find all create/update endpoints that spread request bodies into database writes or bind request objects directly to models. Replace broad binding with explicit allowlisted DTOs/schemas containing only fields the caller may edit. Set sensitive fields on the server from trusted context: owner_id, tenant_id, role, permissions, subscription tier, verified status, balances, price, approval state and timestamps. Reject or ignore unknown/protected fields consistently. Apply validation and authorization per field and per operation. Review nested objects and bulk update endpoints. Add regression tests that submit forged protected properties and confirm they cannot be changed.
```

Required verification: Tests: attempt to set role=admin, another owner_id, verified=true, arbitrary price and subscription state through ordinary user endpoints.

5. Privilege escalation / broken access control

Risk: An ordinary user may reach admin functions, change roles, access another account or bypass object-level permissions.

Copy/paste AI implementation prompt

```
Build an authorization matrix for every route, API, RPC, background job and admin action. Separate authentication (who the caller is) from authorization (what they may do). Enforce checks on the server for every request and every object; never trust hidden UI controls, client-side role flags or user-editable metadata. Use deny-by-default role/permission checks and verify tenant/object ownership. Protect role changes and privileged operations with re-authentication or step-up verification where appropriate, and log them. Review IDOR/BOLA risks by changing resource IDs. Ensure background jobs re-check the initiating user's permissions and scope. Add negative tests for anonymous, normal user, other tenant, revoked user and admin roles.
```

Required verification: Tests: direct API calls with altered IDs; normal-user calls to admin endpoints; revoked-session tests; role-change authorization tests.

6. Open redirects

Risk: An attacker may use a trusted application URL to redirect users to a malicious destination.

Copy/paste AI implementation prompt

```
Find redirect parameters and uses of returnUrl, next, callback, continue and redirect_to. Prefer server-side route identifiers or relative paths. If external destinations are required, validate against a strict allowlist of exact scheme, host and port; reject protocol-relative URLs, user-info tricks, encoded bypasses, backslashes and unexpected schemes. Normalize and parse URLs with a trusted URL parser before validation. Do not use substring or startsWith host checks. Ensure OAuth redirect URIs are exact registered values and validate state/nonce. Add tests for external domains, //host, javascript-like schemes, encoded variants and legitimate internal navigation.
```

Required verification: Tests: malformed and encoded redirect test cases; OAuth callback allowlist verification.

7. Command injection

Risk: Untrusted input passed to an operating-system shell can execute unintended commands.

Copy/paste AI implementation prompt

```
Search for shell execution, process spawning, script runners, archive tools, image converters and system utilities. Remove shell invocation where a library API exists. If a process is unavoidable, call it with a fixed executable and an argument array without shell interpretation; never concatenate user input into a command string. Validate each argument against a strict allowlist, use fixed working directories, minimal environment variables, least-privilege OS accounts, timeouts, output limits and resource limits. Prevent user-controlled executable paths and option injection. Isolate risky processing in a sandbox/container. Add tests with shell metacharacters and option-like values; confirm no command is executed.
```

Required verification: Tests: unit tests around process invocation; verify shell=false/no shell interpolation; sandbox and permission review.

8. Audit logging and monitoring

Risk: Without reliable audit records, suspicious access and important security changes may go undetected or be impossible to investigate.

Copy/paste AI implementation prompt

```
Define security events to record: login success/failure, logout, password and MFA changes, role/permission changes, admin actions, sensitive data exports, payment/subscription changes, API key events, access denials and configuration changes. Record timestamp, actor ID, action, target/resource, outcome, request/correlation ID and minimal context. Never log passwords, tokens, session cookies, full payment data, secret keys or unnecessary personal data. Protect logs from tampering and unauthorized access; use centralized storage, retention rules, alert thresholds and time synchronization. Avoid logging every sensitive payload. Add alerting for repeated failures, privilege changes, unusual exports and anomalous access. Document incident response and log retention.
```

Required verification: Tests: verify required events are emitted; verify secrets are redacted; verify ordinary users cannot read or alter audit logs.

9. Hardcoded secrets

Risk: API keys, database passwords, signing keys or tokens committed to source code can be extracted and abused.

Copy/paste AI implementation prompt

```
Scan tracked files, commit history, build artifacts, source maps, logs and deployment configuration for secrets. Move credentials to the hosting platform's encrypted secret manager or approved environment configuration. Keep privileged credentials server-side; never expose them through NEXT_PUBLIC, VITE_, client bundles or public runtime config. Use separate least-privilege credentials per environment, rotate any exposed key immediately, revoke old credentials, and review access logs for misuse. Add secret scanning to pre-commit and CI; block commits containing detected secrets. Provide a safe .env.example containing placeholders only and ensure real .env files are ignored. Do not print secret values in reports.
```

Required verification: Tests: scan repository and built client assets; verify secret values are absent from bundles, source maps, logs and Git history. Rotation is required if exposure occurred.

10. Weak password hashing

Risk: Fast or unsalted hashes make stolen password databases easier to crack.

Copy/paste AI implementation prompt

```
Inspect password storage and authentication-provider configuration. Prefer managed authentication
with secure password handling. If the application hashes passwords itself, use a modern password
hashing function such as Argon2id with a unique salt and parameters appropriate to current guidance
and available infrastructure; use scrypt or bcrypt where Argon2id is unavailable. Never use plain
SHA-256, MD5 or reversible encryption for passwords. Use constant-time verification where applicable,
secure reset flows, rate limits and breached-password screening where feasible. Plan safe
rehash-on-login upgrades for legacy hashes. Do not invent a custom cryptographic scheme or store
plaintext passwords.
```

Required verification: Tests: verify each password gets a unique salt; inspect algorithm/parameters; verify migration and reset flows; ensure hashes never appear in logs.

11. Slopsquatting / AI-hallucinated dependency names

Risk: An AI agent may suggest a package name that does not exist; an attacker can publish that name with malicious code and wait for developers to install it.

Copy/paste AI implementation prompt

```
Audit dependencies recently added by AI tools and all package manifest/lockfile changes. For each new
package, verify the exact package name, official repository, maintainer history, release history,
registry identity, download/usage signals and whether it is the intended project. Prefer official
packages and trusted namespaces; avoid lookalike names and unverified packages. Pin versions using
the package manager lockfile, use reproducible installs and review install scripts. Use package
allowlists or approval for new dependencies in sensitive projects. Do not blindly install packages
suggested by an LLM. Remove suspicious packages, inspect transitive dependencies and rotate
credentials if a malicious package may have run.
```

Required verification: Tests: clean lockfile install in CI; dependency provenance review; inspect lifecycle scripts and transitive tree. Popularity alone is not proof of safety.

12. Dependency CVEs and supply-chain risk

Risk: Known vulnerabilities in direct or transitive libraries can expose the application even when application code is correct.

Copy/paste AI implementation prompt

```
Run the ecosystem's official dependency audit and a reputable software composition analysis scanner
against manifests and lockfiles. Identify direct and transitive vulnerable packages, severity,
exploitability, reachable code paths and available patched versions. Upgrade to supported patched
releases, regenerate and commit lockfiles, test compatibility, and remove unused dependencies. Avoid
blindly forcing major upgrades. Configure automated dependency update PRs, CI vulnerability gates,
license review and an owner/SLA for critical and high findings. Pin actions and build images where
appropriate. Generate an SBOM for releases and monitor advisories after deployment.
```

Required verification: Tests: clean install; full test/build suite; repeat scanner; document accepted exceptions with owner, rationale and expiry date.

13. Subresource Integrity (SRI) hashes

Risk: Third-party scripts or styles loaded from a CDN may be modified upstream or in transit.

Copy/paste AI implementation prompt

```
Inventory externally hosted script and stylesheet resources. For static, version-pinned resources
that support SRI, add the correct integrity hash and crossorigin attribute. Generate hashes from the
exact deployed asset bytes using a trusted build process; never guess or reuse a hash after the asset
changes. Prefer self-hosting or bundling critical dependencies. Avoid floating CDN versions. Ensure
CSP also restricts allowed sources. Document resources that cannot use SRI, such as dynamically
changing assets, and apply an alternative trusted delivery strategy. Add CI checks for missing or
mismatched integrity metadata.
```

Required verification: Tests: verify the hash matches the exact asset; deliberately alter a test asset and confirm the browser blocks it; ensure deploy updates hashes.

14. Content Security Policy (CSP) headers

Risk: An overly permissive or missing CSP can make cross-site scripting and content injection more damaging.

Copy/paste AI implementation prompt

```
Design a CSP from the application's actual asset, API, image, font, frame and connection origins. Start with report-only mode in staging, collect violations, then enforce a restrictive policy. Prefer nonces or hashes for scripts; avoid unsafe-inline and unsafe-eval unless a documented compatibility requirement exists. Restrict default-src, script-src, object-src, base-uri, frame-ancestors, form-action and connect-src appropriately. Generate per-response nonces securely where used and ensure they cannot be predicted or reused. Do not blindly copy a generic policy. Add other suitable security headers such as X-Content-Type-Options and Referrer-Policy. Document required third-party origins and review them periodically.
```

Required verification: Tests: inspect actual response headers; exercise all app flows; monitor CSP reports; verify unapproved inline scripts and origins are blocked.

15. Clickjacking

Risk: An attacker may embed the application in a frame and trick a user into clicking hidden controls.

Copy/paste AI implementation prompt

```
Prevent framing of sensitive application pages using CSP frame-ancestors with a restrictive policy, normally 'none' unless legitimate embedding is required. For legacy browser support, configure X-Frame-Options DENY or SAMEORIGIN consistently. If selected pages must be embeddable, allow only explicitly trusted origins and test them. Do not rely on JavaScript frame-busting as the primary defense. Review OAuth, payment and embedded-widget flows for compatibility. Apply headers at the server/CDN layer so all relevant routes receive them.
```

Required verification: Tests: inspect headers on authenticated and unauthenticated pages; attempt to embed a test page from an untrusted origin.

16. Public storage bucket / exposed object storage

Risk: Private user uploads, invoices, reports or identity documents may be readable through public buckets or predictable URLs.

Copy/paste AI implementation prompt

```
Inventory every storage provider, bucket/container, access policy, object ACL, CDN origin and signed URL flow. Make private data private by default. Restrict read/write access using server-side authorization and tenant ownership; never rely on obscurity of object paths. Use short-lived signed URLs for authorized downloads, with narrowly scoped object and method permissions. Validate upload type, size and content; generate safe object names; scan files where appropriate; prevent executable content from being served unsafely. Review backups and derived thumbnails too. Remove public access where not explicitly required and inspect access logs for prior exposure.
```

Required verification: Tests: unauthenticated access denied; user A cannot fetch user B's object; expired signed URL fails; public assets remain intentionally accessible.

17. Debug mode and verbose error exposure

Risk: Debug pages, stack traces and development endpoints can reveal secrets, source paths, SQL, environment variables and internal architecture.

Copy/paste AI implementation prompt

```
Ensure production builds explicitly disable framework debug mode, verbose exception pages, development servers, hot reload endpoints, test routes and diagnostic consoles. Return generic client-facing errors with a correlation ID; keep detailed diagnostics in access-controlled server logs with secrets redacted. Remove sample credentials and test accounts. Verify source maps are handled intentionally and not publicly exposed if they contain sensitive source. Confirm production environment selection cannot be changed by a client request. Add deployment checks that fail if debug flags or unsafe diagnostic routes are enabled.
```

Required verification: Tests: trigger controlled errors in staging and inspect responses; scan deployed routes and assets for debug endpoints and secrets.

18. Token expiry, revocation and session security

Risk: Long-lived or poorly validated tokens can remain usable after logout, compromise or permission changes.

Copy/paste AI implementation prompt

```
Inventory access tokens, refresh tokens, password reset tokens, email verification tokens, API keys and browser sessions. Enforce issuer, audience, signature algorithm, expiry, not-before and intended token type validation. Use short-lived access tokens and securely rotated refresh tokens with reuse detection where supported. Revoke sessions on logout, password reset, account compromise and relevant privilege changes. Store browser session credentials in appropriately protected cookies (Secure, HttpOnly, suitable SameSite) where the architecture supports them; protect state-changing requests against CSRF. Avoid storing long-lived credentials in localStorage. Apply idle and absolute session limits based on risk. Use cryptographically secure random tokens and store only hashes of one-time tokens when feasible.
```

Required verification: Tests: expired, malformed, wrong-audience, revoked and reused refresh tokens fail; logout invalidates session; reset invalidates appropriate sessions.

19. Insecure deserialization

Risk: Deserializing untrusted data into executable or privileged object types can cause code execution, type confusion or authorization bypass.

Copy/paste AI implementation prompt

```
Find deserialization of client input, cookies, queue messages, cache entries, uploaded files and third-party responses. Avoid native object deserialization for untrusted data. Prefer simple JSON with strict schema validation, explicit types, size/depth limits and allowlisted fields. Disable polymorphic type metadata and automatic class instantiation unless there is a compelling, reviewed need. Verify message authenticity and authorization independently of serialized contents. Treat signed data as integrity-protected, not automatically authorized. Reject malformed, oversized, deeply nested and unexpected fields safely. Review language-specific serializers and unsafe YAML, pickle, PHP object or Java native serialization usage.
```

Required verification: Tests: malformed and unexpected-type payloads are rejected; size/depth limits work; no attacker-controlled class instantiation occurs.

20. Server-Side Request Forgery (SSRF) — recommended addition

Risk: The screenshot leaves item 20 blank. SSRF is added here as a recommended twentieth check because server-side URL fetchers, webhooks, importers and preview tools are common attack surfaces.

Copy/paste AI implementation prompt

```
Inventory every server-side HTTP request feature: URL previews, image fetchers, PDF generation, webhook testers, imports, integrations and AI browsing tools. Do not fetch arbitrary user-provided URLs without controls. Prefer strict hostname allowlists. Parse and normalize URLs with a trusted parser; allow only required schemes (normally HTTPS), reject credentials in URLs and unexpected ports, and block loopback, private, link-local, multicast and cloud metadata IP ranges for both IPv4 and IPv6. Resolve DNS and validate the resolved IP; defend against DNS rebinding and redirects by re-validating every redirect target and limiting redirect count. Apply egress firewall rules, timeouts, response-size limits, content-type checks and network isolation. Do not forward internal cookies, authorization headers or cloud credentials. Where possible, route outbound requests through a controlled proxy.
```

Required verification: Tests: reject localhost, private IPs, metadata endpoints, encoded IP forms, IPv6 local addresses and redirects to blocked networks; verify egress restrictions at infrastructure level.

Additional remediation: exposed .git directory

The second screenshot highlights a separate deployment misconfiguration: a web server must not serve the Git metadata directory. Blocking `/.git` is necessary, but the repository must also be removed from the public deployment artifact.

```
Inspect the deployment pipeline, build output, hosting configuration, reverse proxy and web server
rules. Ensure the production artifact contains only files required to run the application; do not
deploy the repository's .git directory, Git objects, refs, config, hooks, index or pack files. Add
.git, .git/** and relevant local environment files to deployment exclusions where supported, but do
not assume .gitignore controls deployment uploads. For Nginx, add a location rule that denies access
to any path beginning with /.git (including nested paths), returning 403 or 404 without serving file
contents; ensure regex/location precedence cannot bypass it. Apply equivalent deny rules at the CDN,
Apache, Caddy or hosting layer if used. Do not expose directory listings. Check that source archives,
backups, static hosting roots and alternate hostnames do not publish repository metadata. If Git
metadata or secrets were publicly exposed, treat credentials as compromised: rotate them, review
access logs and consider whether source history contains secrets. Do not simply rename .git or rely
on robots.txt.
```

Nginx example (adapt and test in the actual server configuration)

```
# Deny Git metadata paths; place in the correct server block.
location ~* (^|/)\.git(/|$) {
deny all;
return 404;
}
```

Deployment verification

- Inspect the deployed document root/build artifact and confirm no .git directory or Git metadata is present.
- Request /.git/HEAD, /.git/config, /.git/index and a nested path such as /assets/.git/HEAD. They must not return repository contents.
- Check URL-encoded and case variants where relevant to the server/platform; confirm normalization cannot bypass the deny rule.
- Check alternate domains, preview deployments, backup archives and CDN origins.
- Remember: a 403/404 response is a useful check, but it does not prove .git was excluded from the artifact. Verify both deployment contents and HTTP behavior.

Example Nginx configuration is not universal. Validate with `nginx -t`, reload safely, and verify the effective configuration and deployed behavior. Do not paste commands into production without understanding the hosting environment.

Master production security audit prompt

```
Act as an independent application security reviewer. Audit this complete application against all 20
controls in this document plus exposed repository metadata. First map architecture, trust boundaries,
data stores, identity provider, roles, tenant model, APIs, storage, LLM tools, deployment pipeline
and external integrations. For each control, classify status as PASS, FAIL, PARTIAL, NOT APPLICABLE
or NEEDS MANUAL VERIFICATION. Cite exact files/routes/configuration and explain the evidence. Do not
mark a control PASS merely because a library or header exists; verify it is active on the real
execution path. Prioritize findings by severity and exploitability, identify affected data and likely
impact, and propose minimal fixes. Implement approved fixes in a branch, add regression tests, run
unit/integration/security scans and build checks, then produce a retest report. Include unresolved
risks, owner, remediation deadline, provider dashboard settings, infrastructure changes, secret
rotation steps and rollback plan. Never claim the application is secure or production-ready if any
critical/high issue remains unverified.
```

Release gate checklist

- ■ All 20 controls reviewed and status/evidence recorded; no critical or high findings left open without explicit risk acceptance.
- ■ Cross-user and cross-tenant authorization tests pass for every sensitive resource.

- ■ No production secrets in source, client bundles, source maps, logs, Git history or public artifacts; exposed credentials rotated.
- ■ RLS and storage access policies tested with multiple real roles and anonymous access.
- ■ Authentication, password reset, MFA (if offered), session expiry, revocation and CSRF protections verified.
- ■ Dependencies scanned; lockfile committed; critical/high CVEs remediated or formally risk-accepted.
- ■ Security headers, CSP, framing controls, HTTPS and cookie attributes verified on deployed responses.
- ■ Uploads, deserialization, outbound URL fetches, webhooks and background jobs have limits and authorization checks.
- ■ Audit logs are useful, access-controlled, tamper-resistant and do not contain secrets or excessive personal data.
- ■ Production debug mode disabled; errors sanitized; monitoring and alerting tested.
- ■ .git and environment files excluded from deployment; direct HTTP probes return no repository content.
- ■ Backups, restore procedure, incident response contacts, retention/deletion rules and rollback plan documented.
- ■ Staging verification completed; independent review scheduled for sensitive/high-risk applications.

Useful standards and references

OWASP Top 10 — https://owasp.org/www-project-top-ten/

OWASP ASVS — https://owasp.org/www-project-application-security-verification-standard/

OWASP Cheat Sheet Series — https://cheatsheetseries.owasp.org/

OWASP API Security Top 10 — https://owasp.org/www-project-api-security/

NIST Secure Software Development Framework (SSDF) — https://csrc.nist.gov/Projects/ssdf

Mozilla HTTP Observatory / web security guidance — https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers

Scope note: the screenshots show 19 named checklist items and an empty number 20. SSRF is explicitly marked as an added recommendation, not represented as text from the screenshot. The Git exposure fix is included separately because it appears in the second screenshot.